

Отчет по выполненной работе:
“Векторизация КТ-данных”

Выполнил: Фейгенберг М.И.
Проверил: Полевой Д.В.
Группа: БПМ-23-ПО-3, 3 курс

Москва, 2026

Постановка задачи.....	3
Описание алгоритма.....	4
Предобработка.....	4
Сегментация.....	5
Скорлупа.....	5
Область интереса.....	7
Ядро.....	8
Перегородки.....	11
Построение полигональных сеток.....	13
Морфологическое сглаживание 3D-объема.....	14
Генерация сетки.....	14
Постобработка и удаление шипов.....	14
Экспорт сетки.....	15
Запуск программы.....	16
Оценка результатов.....	17
Визуальная оценка качества.....	17
Метрики точности сегментации.....	18
Анализ ошибок.....	19
Неточности маски перегородок.....	19
Проблемы с маркерами ядра для Watershed.....	20
Неточности в полигональных сетках.....	22

Постановка задачи

Дан массив последовательных 500 КТ-сканов грецкого ореха с шагом в 1мм (см рис. 1).

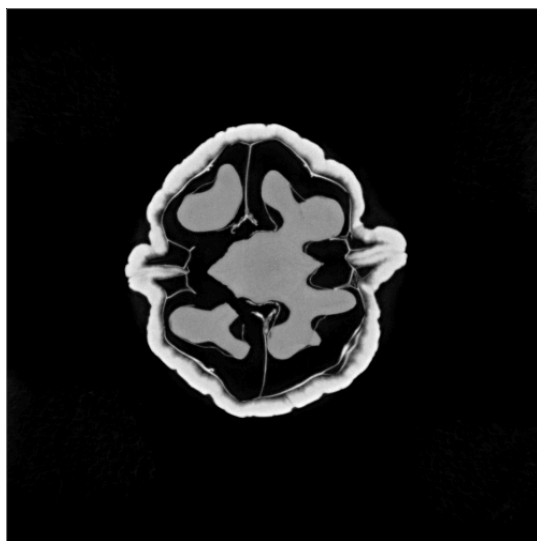


Рис. 1, 305 КТ-скан

Стоит задача векторизации – приведения данного массива к набору 3-х мерных вершин и граней каждого из классов содержимого ореха: скорлупы, перегородок и ядра ореха. Выполнение этой задачи можно разделить на два основных этапа:

1. Сегментация исходного массива вокселей на три класса для создания масок объемов (см рис. 2)

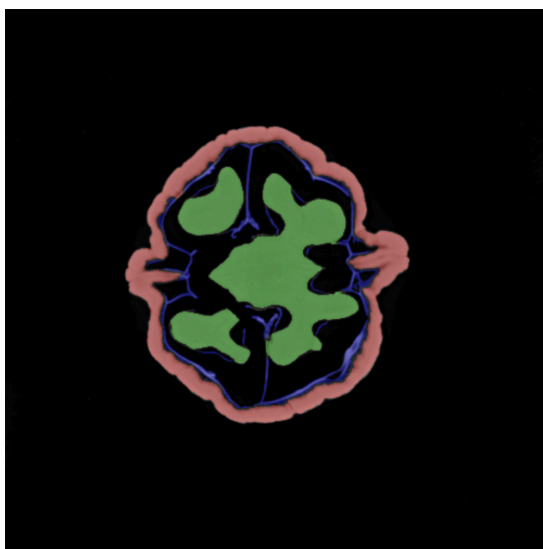


Рис. 2, Оверлей полученных сегментацией масок классов на 305 КТ-скане

2. Построение полигональных сеток (см рис. 3) для каждого из объемов.

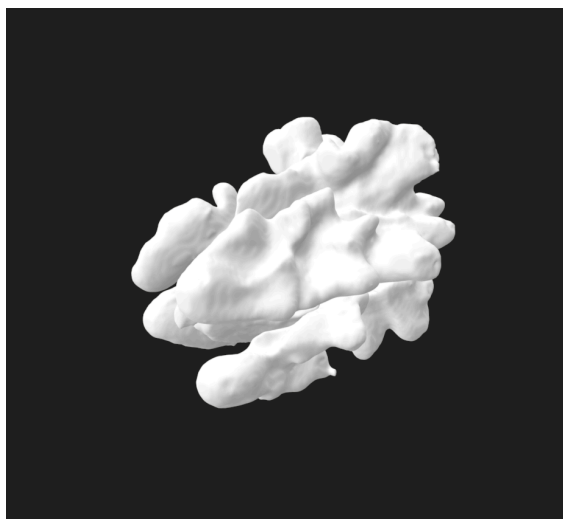


Рис. 3, Полигональная сетка ядра ореха

Описание алгоритма

Предобработка

В исходном массиве 500 КТ-сканов были выделены начальные и конечные координаты грецкого ореха по трем осям для отсеечения неинформативных слайсов, которые могут внести ненужный шум и помешать корректной работе алгоритма сегментации. В итоге полезными по оси Z оказались 411 слайсов, оси X - 299, Y - 326. Ограничение по трем осям выставляются вручную через программу **bounds_gui.py** (см рис. 4), json на выходе из который позже передается алгоритму сегментации.

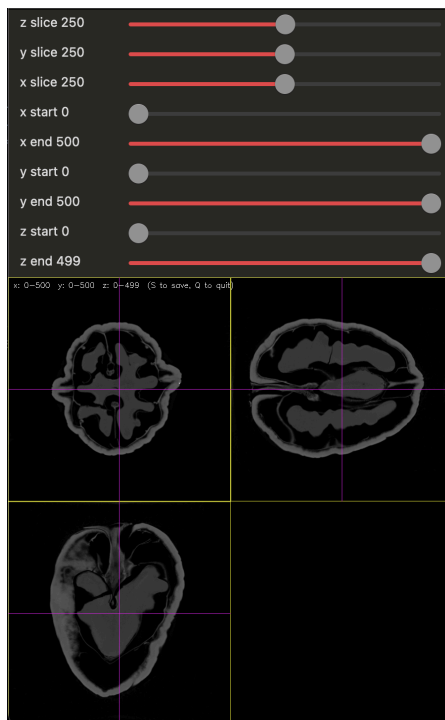


Рис. 4, Программа bounds_gui.py

Срезы автоматически загружаются и объединяются в единый объем в классе **VolumeStore**. Перед сегментацией по всем срезам вычисляется глобальный максимум и минимум яркости для нормализации и контрастирования. Итоговый объем нормализуется по формуле: $img8 = (img - min) / (max - min) * 255$, после чего контрастируются умножением на 2 и клиппингом от 0 до 255 (см рис. 5).

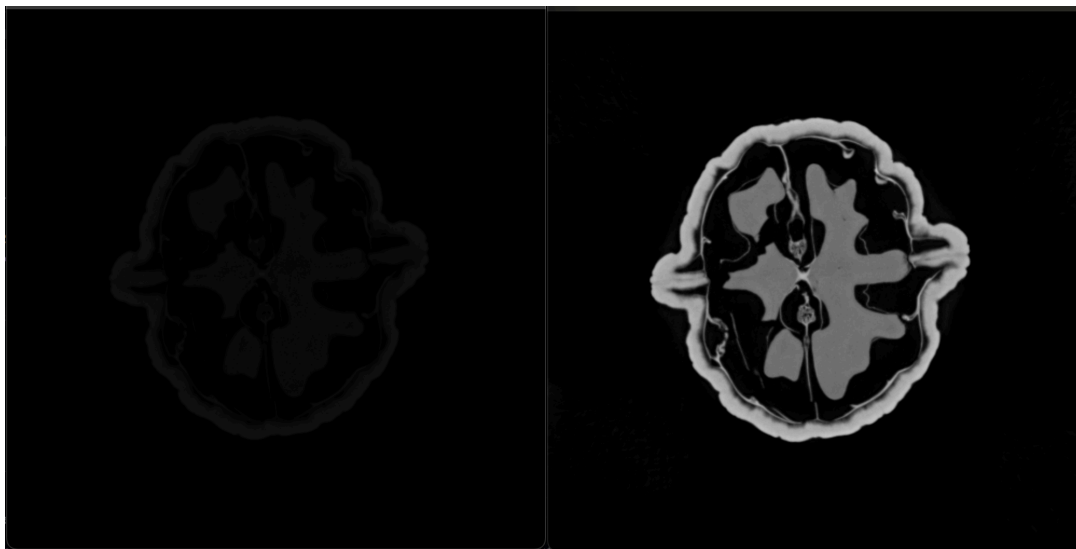


Рис. 5, Результат предобработки массива на примере 205 слайса по Z

Сегментация

Сегментация выполняется в классе **Segmentator** и включает последовательные шаги, описанные ниже. Порядок шагов строго задан: каждый шаг опирается на результат предыдущего. Алгоритм обрабатывает воксельный объем, представленный классом **VolumeStore**, и позволяет задавать пространственные границы ореха для отсечения фонового шума по краям скана.

Скорлупа

Скорлупа сегментируется в три прохода: вдоль осей Z, X и Y (см рис. 6). Это позволяет достичь более высокой точности сегментации и забирать нераспознанные при одном проходе части скорлупы.

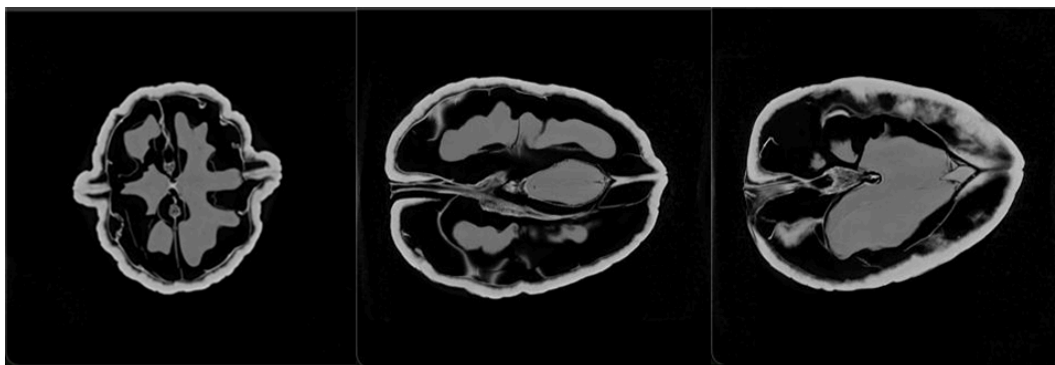


Рис. 6, Срезы середины ореха по осям Z, X, Y

1. Для каждого среза вычисляется гистограмма яркостей. Порог яркости для бинаризации ядра вычисляется модифицированным методом Оцу, где поиск порога начинается не с нуля, а со определенного заданного значения, которое устанавливается равным $img.mean() * 1.3$. Применяя такой подход, мы можем исключить около-нулевые пиксели фона из гистограммы, гарантируя, что порог будет найден между классами ореха (более яркой скорлупой и тусклыми перегородками с ядром), а не между задним фоном и самим орехом.

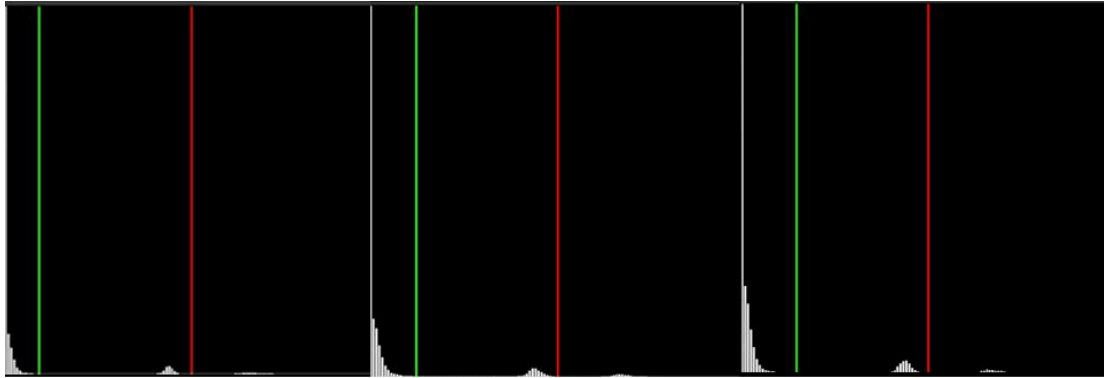


Рис. 7, Гистограммы срезов с рис. 6, зеленым - начальный порог, красным - порог Оцу

2. Пиксели выше порога бинаризуются, применяется морфологическое открытие для удаления мелкого шума.



Рис. 8, Результат бинаризации срезов с рис. 7 по порогу Оцу

3. Далее алгоритм избавляется от мелкого шума и объединяет несколько отдельных частей скорлупы в единую маску, если необходимо. Анализируется история площадей масок скорлупы на предыдущих 7 срезах, считая среднюю площадь и среднее соотношение соседних масок, после чего с текущего среза адаптивно отбираются только те связные компоненты, чья суммарная площадь близка к ожидаемой.
4. Тонкие компоненты (например, остатки перегородок) удаляются на основе дистанционного преобразования - компоненты с небольшой толщиной (меньше 7px) и небольшой площадью (меньше 20px).



Рис. 9, Очищенная и объединенная бинарная маска с рис. 7

5. Для уточнения границ применяется алгоритм Watershed на дополнительно контрастированном изображении (для лучшего отделения границ скорлупы от перегородок). В качестве достоверного фона используется расширенная маска скорлупы, полученная на прошлом шагу, а в качестве маркеров объектов - пиксели внутри маски, полученные с помощью дистанционного преобразования, по порогу *0.1 максимальной толщины маски*.

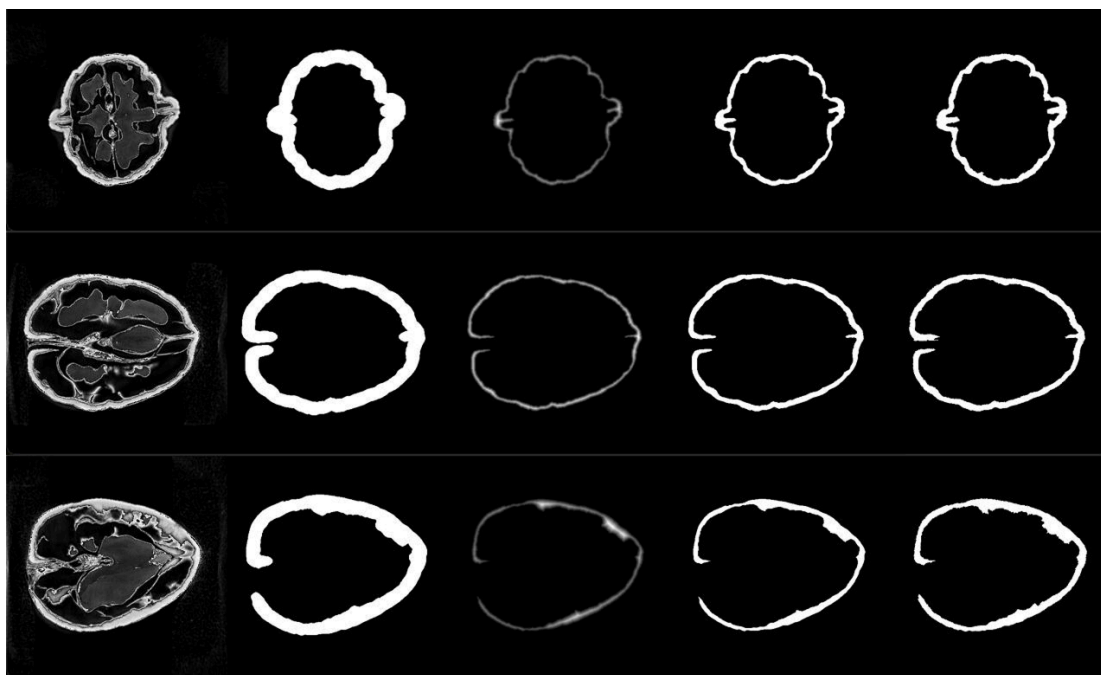


Рис. 10, Дополнительно контрастированное изображение, маска уверенного фона, дистанционное преобразование, маркеры скорлупы для Watershed и итоговая маска для центральных срезов по Z, X, Y

6. Итоговые маски с трех осей объединяются в одну методом голосования (выбираются воксели, отмеченные минимум на 2 из 3 проекций). Применяется 3D-морфологическое замыкание.

Область интереса

Для локализации области интереса (внутренностей скорлупы) - применяется метод `expand_outside()`, описанный внутри `VolumeStore`. Метод заливает фон на каждом слайсе с помощью `cv2.floodFill`, превращая все, что вне скорлупы, в белый цвет, после

чего маска окрестностей скорлупы объединяется с самой маской скорлупы, чтобы получить инвертированную маску внутренностей ореха.

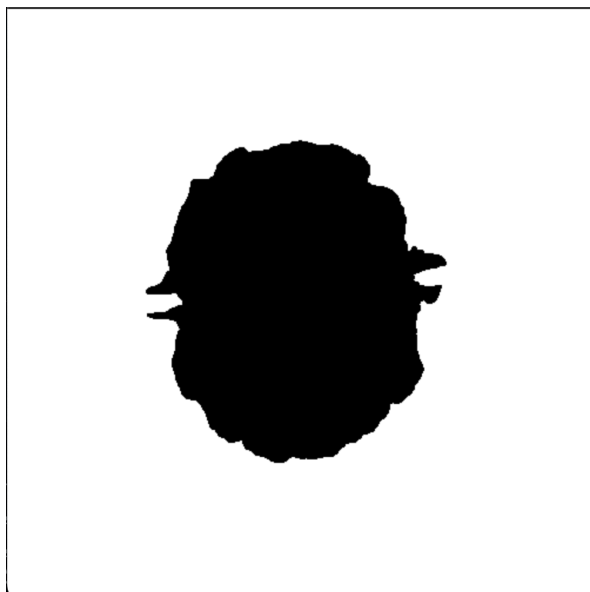


Рис. 11, Инвертированная маска внутренностей скорлупы для центрального среза по Z. Затем из исходного КТ-объема исключаются (выставляются в 0) пиксели, которые оказались вне внутренностей ореха, с помощью метода `subtract()`, описанного в *VolumeStore*. Для предотвращения попадания пикселей границы скорлупы во внутренности, маска скорлупы предварительно расширяется на радиус 2. В итоге получается объем, содержащий только ядро и перегородки.



Рис. 12, Полученная вычитанием скорлупы внутренность скорлупы

Ядро

Сегментация ядра выполняется внутри полученного ROI.

1. Как и для скорлупы, применяется адаптивный метод Оцу для бинаризации.



Рис. 13, Гистограмма внутренностей ореха (красным отмечен порог Оцу) и бинарная маска внутренностей

2. Полученная маска эродируется для разделения слипшихся долей ядра и прилипших к ним перегородок.

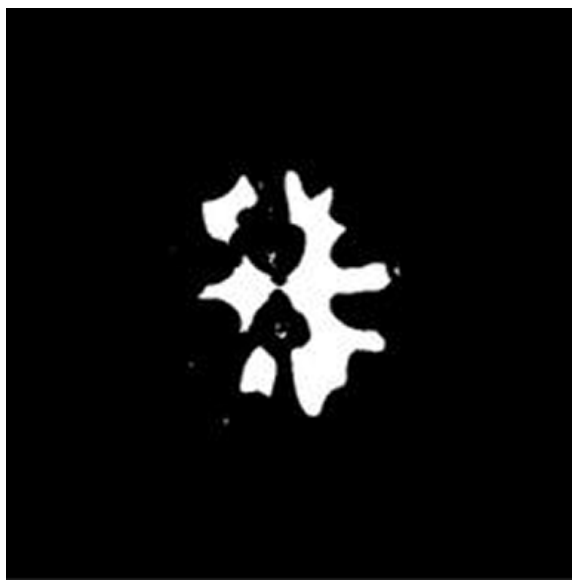


Рис. 14, Маска внутренностей ореха после эрозии

К ней применяется дистанционное преобразование. Пиксели со значением больше $0.3 * \max$ становятся маркерами ядра для алгоритма Watershed.



Рис. 15, Дистанционное преобразование и маркеры ядра `sure_fg`

Порог в 0.3 максимального радиуса устанавливался в соответствии с соотношением малых и больших ядер на одном срезе.

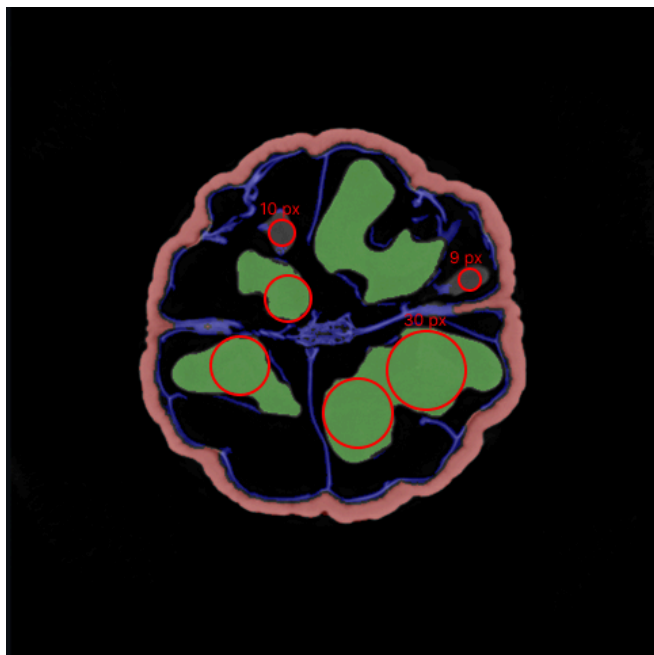


Рис. 16, Оценка радиусов ядер и их соотношений на 152 срезе по Z

3. В качестве маркера фона используется дилатированная бинарная маска (см рис. 13)



Рис. 17, Маркеры фона

4. Далее на дополнительно контрастированном изображении (см рис. 18) для лучшего отделения перегородок от ядра запускается алгоритм Watershed, чтобы уточнить границы ядра на срезе.

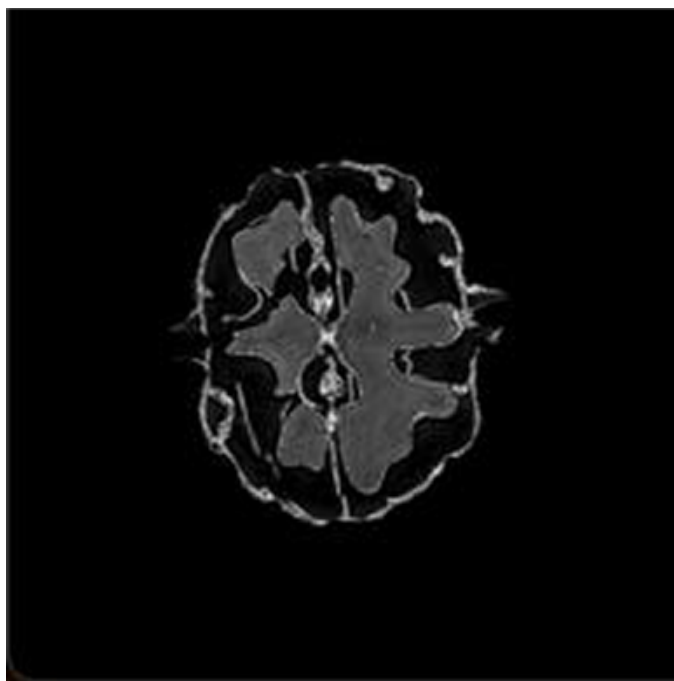


Рис. 18, Контрастированное изображение для алгоритма Watershed



Рис. 19, Полученная алгоритмом Watershed маска ядра

5. После обработки всех срезов применяется 3D-анализ связанных компонент. Компоненты с объемом менее 3000 вокселей удаляются как шум. Применяется морфологическое замыкание (радиус 3) для заполнения дыр.

Перегородки

Перегородки - самые тонкие и сложные структуры внутри ореха (они могут быть самыми разными по толщине и по яркости в разных местах). Из ROI предварительно вычитается маска ядра (с защитным радиусом 3px).



Рис. 20, ROI с вычтенной маской ядра

1. К изображению применяется морфологическая операция Тор-Нат с эллиптическим ядром 9x9. Это позволяет убрать большие остатки ядер и скорлупы и оставить на изображении только тонкие перегородки, размер 9x9 был подобран экспериментально.



Рис. 21, Результат работы ТорНат

2. Вычисляются два порога: сильный методом Оцу и слабый ($t_{strong} * 0.5$).

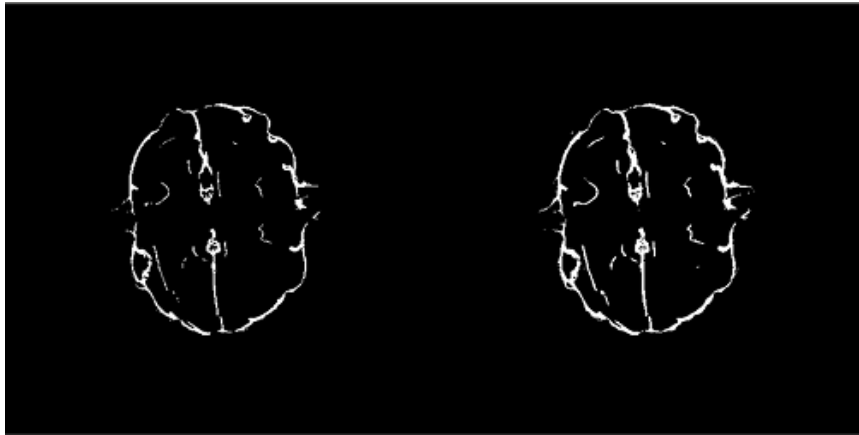


Рис. 22, Бинарные маски от сильного и слабого порогов

3. Пиксели выше слабого порога формируют множество кандидатов. Алгоритм оставляет только те связные компоненты кандидатов, которые содержат хотя бы один пиксель из сильного порога (метод гистерезиса). Это позволяет сохранить тонкие перегородки, отбрасывая шум.

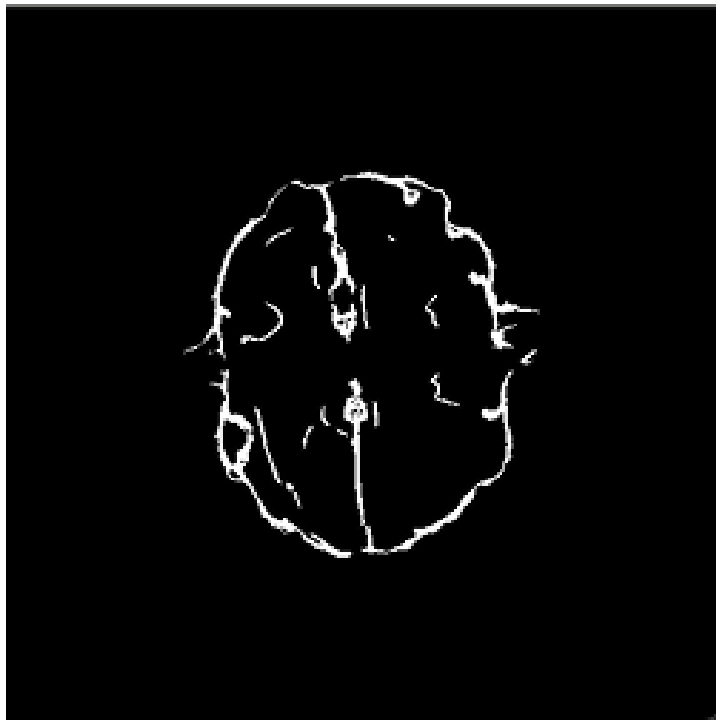


Рис. 23, Результат сегментации перегородок

Построение полигональных сеток

Код строит 3D-модель грецкого ореха и экспортирует результаты в формат OBJ. За генерацию отвечает класс `Meshes`, который в параметры принимает `VolumeStore` с масками классов, полученных на этапе сегментации.

Морфологическое сглаживание 3D-объема

Функция `smooth_volume()` отвечает за предобработку воксельного объема. Поскольку при сегментации между соседними срезами могут возникать ступеньки или мелкие разрывы, применяется операция морфологического закрытия с использованием сферического структурного элемента (радиус 2). Это заполняет микро пустоты и делает объект сплошным перед генерацией сетки.

Генерация сетки

Преобразование бинарного 3D-массива в полигональную сетку происходит в функции `volume_to_mesh()`. Используется алгоритм Marching Cubes. Он обходит воксели и строит триангулированную изоповерхность на уровне *level=0.5*. Выбор уровня 0.5 разумен, так как массив приводится к булеву типу, где 1 — объект, а 0 — воздух.

Параметр *spacing = (0.1, 0.1, 0.1)* масштабирует координаты вершин в физические размеры (миллиметры). Для оптимизации производительности используется параметр *step_size=2*, который пропускает каждый второй воксель.

Постобработка и удаление шипов

На этапе постобработки (`postprocess_mesh()`) качество сетки улучшается:

1. Удаление шипов: Функция `remove_spikes()` вычисляет длину всех уникальных ребер сетки. Если длина ребра превышает *spike threshold mm* (для перегородок это 0.3 мм), грани, образующие это ребро, удаляются. Это убирает одиночные торчащие треугольники артефакты от Marching Cubes.
2. Сглаживание: Применяется фильтр Лапласа с параметрами *lamb=0.5* и *iterations=5*. Фильтр перемещает каждую вершину ближе к геометрическому центру ее соседей, сглаживая лесенку от вокселей.

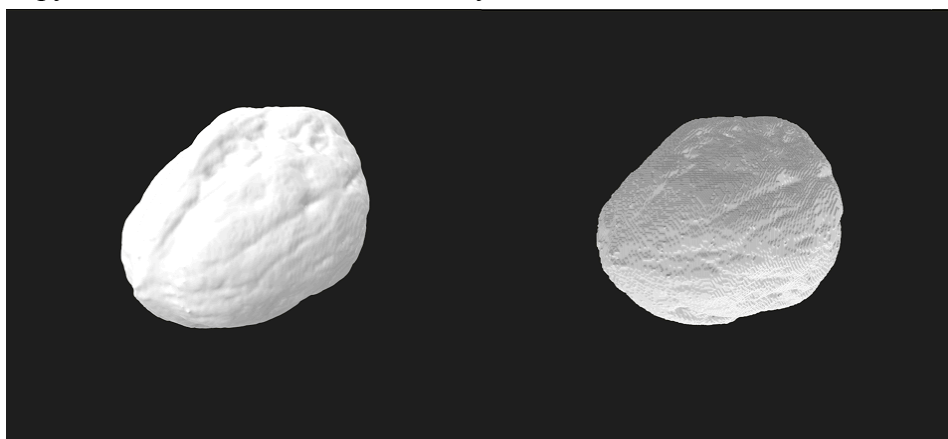


Рис. 24, Сетка скорлупы ореха после и до сглаживания

3. Для перегородок дополнительно вызывается функция `mesh.split()`, которая оставляет только крупнейшую связную полигональную компоненту, отбрасывая плавающий в пространстве шум.

Экспорт сетки

Функция `save_mesh()` сохраняет итоговый объект в формат *.obj*. Файлы сохраняются в директорию `output/meshes/` с именами *walnut_shell.obj*, *walnut_kernel.obj* и *walnut_septa.obj*.

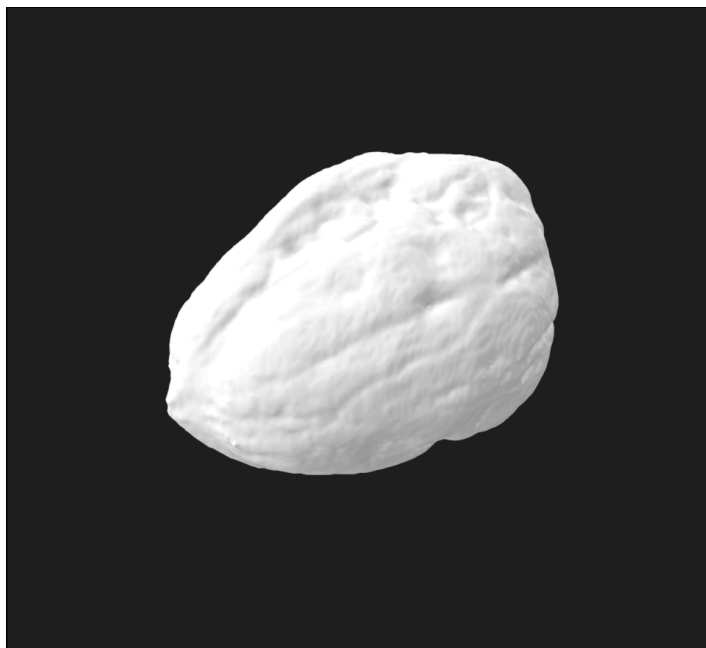


Рис. 25, Итоговая полигональная сетка скорлупы ореха

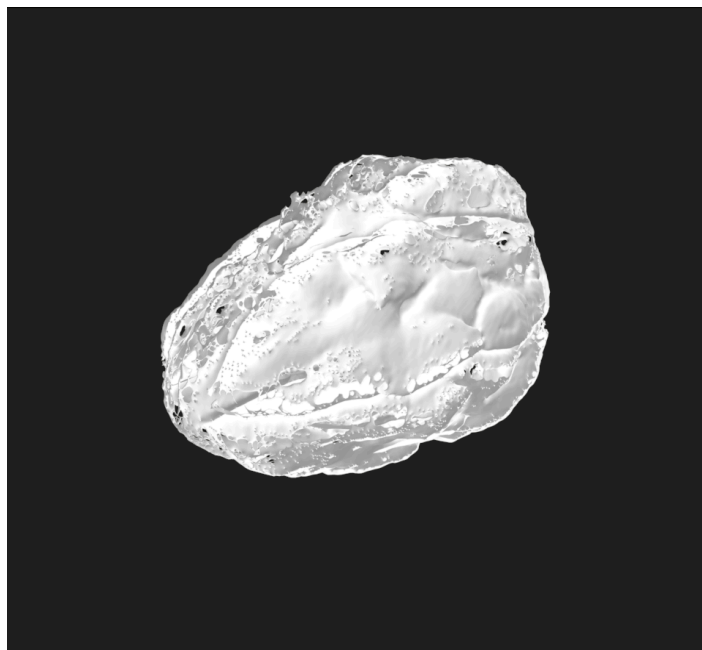


Рис. 26, Итоговая полигональная сетка перегородок

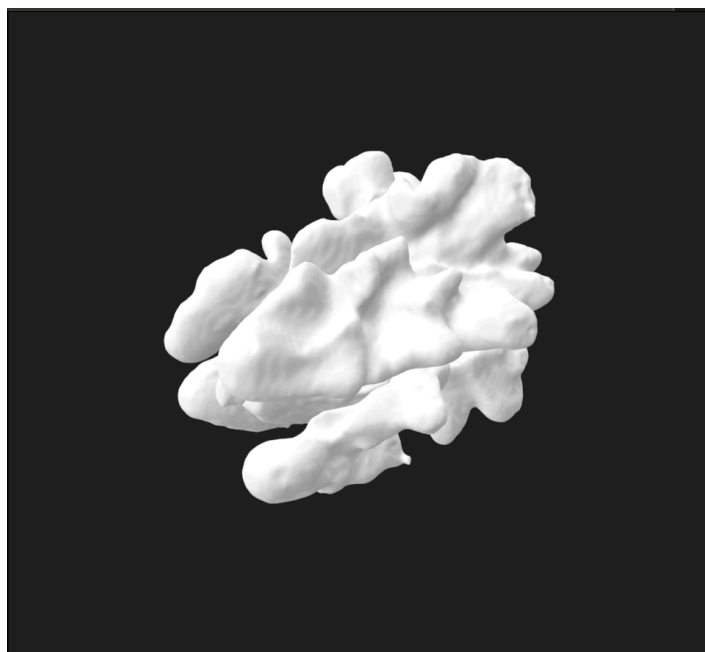


Рис. 27, Итоговая полигональная сетка ядра ореха

После всех шагов очистки построены три полигональные сетки. Их размерности приведены в таблице ниже:

	Вершины	Полигоны
Скорлупа	283 825	567 706
Перегородки	370 105	771 282
Ядро	138 598	277 252

Таблица 1, размерности итоговых полигональных сеток

Запуск программы

Для воспроизведения результатов необходимо запускать *main.py* из корня проекта через следующий команду:

uv run main.py

Для корректной работы алгоритма необходимо указать следующие флаги:

1. `--folder` - папка с исходными срезами
2. `--output` - выходная директория (по умолчанию создается папка `output` в корне программы)
3. `--bounds` - путь к JSON-файлу с границами ореха
4. `--show-debug` - позволяет визуализировать промежуточные этапы (Оцу, Watershed, Top-Hat) для центрального среза.
5. `--evaluate` - позволяет провести оценку качества результатов сегментации на сравнении с эталонными масками (необходимо указать путь к эталонным маскам через флаг `--etalon-dir`)

Для запуска программы выбора границ ореха необходимо запустить `bounds_qui.py` из корня проекта через следующую команду:

uv run lab/helpers/bound_qui.py

Для корректной работы программы необходимо указать следующие флаги:

1. --folder - папка с исходными срезами
2. --ext - расширение исходных срезов

Оценка результатов

Визуальная оценка качества

Наложение масок на исходный срез позволяет быстро обнаружить грубые ошибки: скорлупа - красная, ядро - зелёное, перегородки - синие. Оверлей используется для подбора значений параметров запуска на исходном массиве и грубой оценки качества текущей реализации сегментации.

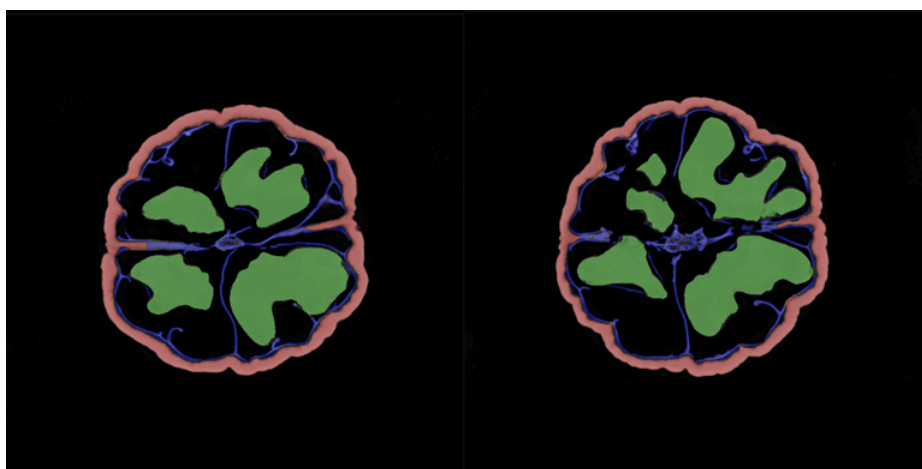


Рис. 28, Визуализация сегментации на слоях 137, 160 по Z

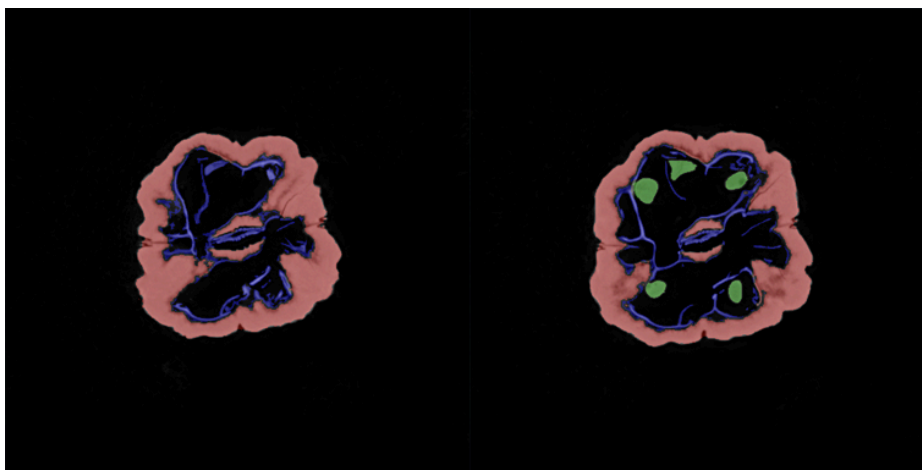


Рис. 29, Визуализация сегментации на слоях 72, 79 по Z

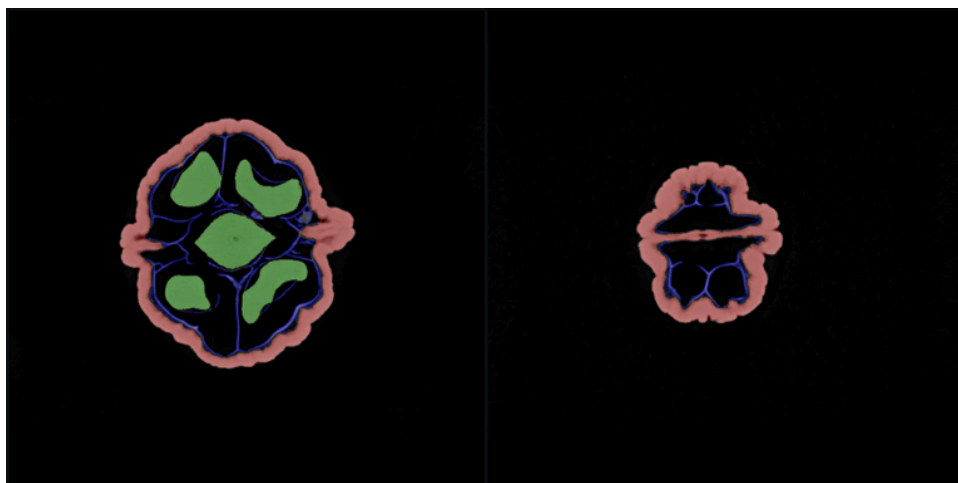


Рис. 30, Визуализация сегментации на слоях 329, 397 по Z

Смотря на наложения масок оверлеем можно сказать, что в общем алгоритм в большинстве случаев срабатывает правильно, большинство пикселей исходного изображения корректно маркированы соответствующей маской.

Метрики точности сегментации

Из общего набора в 411 среза по оси Z были выбраны 21 слайс, для которых была проведена эталонная разметка. Сами слайсы были выбраны, начиная с 40 по 440 с шагом в 20.

Для количественной оценки качества работы алгоритма реализован класс **SegmentationQualityEvaluator**. Он сравнивает полученные маски с эталонными изображениями, загружаемыми из указанной директории.

Для каждого среза вычисляются классические метрики бинарной классификации:

1. IoU (Intersection over Union) - отношение площади пересечения масок к площади их объединения.

$$\frac{TP}{TP + FP + FN} \quad (1)$$

2. Dice - коэффициент сходства, равный:

$$\frac{2 * TP}{2 * TP + FP + FN} \quad (2)$$

3. Precision и Recall - доли правильно определенных пикселей относительно всех предсказанных и всех эталонных соответственно.

Скрипт **main.py** при запуске с флагом **--evaluate** усредняет эти метрики по всем доступным эталонным срезам и выводит стандартное отклонение между метриками всех доступных срезов, что позволяет оценить стабильность работы алгоритма на разных срезах.

По результатам сравнения с эталонными масками, которые можно посмотреть [здесь](#), получаются средние метрики качества:

	Ядро	Скорлупа	Перегородка	Всего
IoU $\pm$ std	0.92 ± 0.06	0.83 ± 0.03	0.54 ± 0.19	0.76 ± 0.09
Dice $\pm$ std	0.96 ± 0.03	0.90 ± 0.02	0.68 ± 0.16	0.85 ± 0.07
Precision $\pm$ std	0.99 ± 0.01	0.98 ± 0.03	0.60 ± 0.18	0.86 ± 0.07
Recall $\pm$ std	0.93 ± 0.06	0.84 ± 0.05	0.79 ± 0.15	0.85 ± 0.08

Таблица 2, Оценка качества сегментации КТ сканов грецкого ореха на 21 слайсе по Z

Тестирование разработанного алгоритма сегментации, проведенное на выборке из 21 КТ-срезов грецкого ореха, продемонстрировало хорошую общую эффективность (среднее значение Dice = 0.85, IoU = 0.76). Наилучшая точность сегментации достигнута для ядра (Dice = 0.96, IoU = 0.92), что обусловлено крупными размерами и четкими границами данной структуры. Сегментация скорлупы также показала уверенные результаты (Dice = 0.90, IoU = 0.83). В то же время для перегородок зафиксированы средние значения метрик (Dice = 0.68, IoU = 0.54). Снижение показателей является ожидаемым: тонкая и сильно разветвленная морфология перегородок делает их критически чувствительными к пространственным смещениям. Из-за малой площади, занимаемой этой структурой, даже незначительные отклонения предсказанной маски от эталона приводят к существенному падению значений метрик, мы можем заметить что полнота выделения перегородок находится на достаточно хорошем уровне, при этом точность страдает, следовательно мы можем сделать вывод что алгоритм слишком много забирает в маску перегородок.

Анализ ошибок

Неточности маски перегородок

Перегородки оказались наиболее сложным классом для точной сегментации по нескольким причинам:

1. Находятся близко как к скорлупе, так и к ядру
2. Неоднородны по яркости
3. Площадь перегородок меняется быстро от слайса к слайсу

По значениям IoU можно было наблюдать, что различие с эталонной разметкой наибольшее, чем на других структурах. Алгоритм выделяет перегородки на итоговой маске толще, нежели на эталонных и в некоторых случаях упускает малые части перегородки, из за чего, в купе с малой площадью самих перегородок, сильно бьется статистика. Если оценивать геометрически полученные маски перегородок на сравнении с эталонными, они имеют крайне схожую структуру. Для уточнения итоговых масок, возможно, стоит применить сглаживание и очистку от мусора (ни к чему не присоединенных частей маски) к полученному массиву.

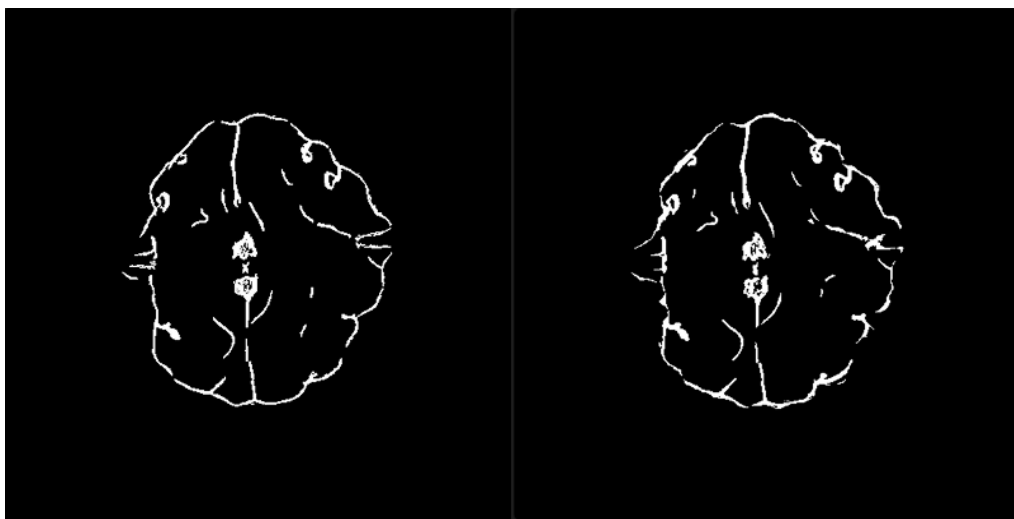


Рис. 31, Сравнение эталонной маски перегородок и полученной сегментацией на слайсе 200 по Z

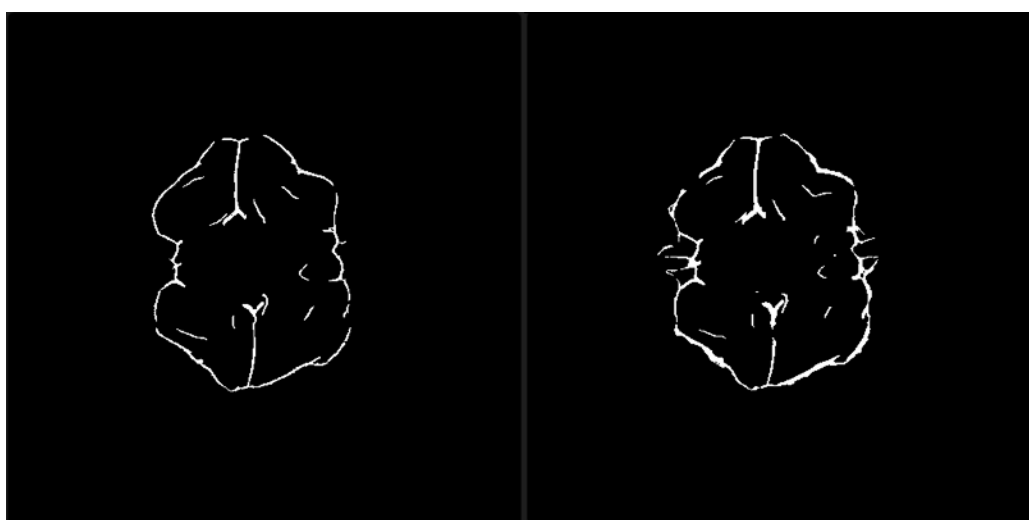


Рис. 31, Сравнение эталонной маски перегородок и полученной сегментацией на слайсе 300 по Z

Проблемы с маркерами ядра для Watershed

Применение алгоритма Watershed для восстановления формы долей и уточнения границ ядра ограничено его зависимостью от входных маркеров. Сбой сегментации возникает при потере маркеров, что происходит в двух сценариях:

- избыточное удаление структуры при эрозии (на срезах с малой площадью ядра)
- фильтрация маркера на этапе дистанционной трансформации как слишком мелкого.

Это приводит к нестабильной работе алгоритма, которая проявляется в виде ложного исчезновения маски ядра на краевых срезах (по оси Z) и на срезах с существенной разницей в размерах фрагментов ядра.

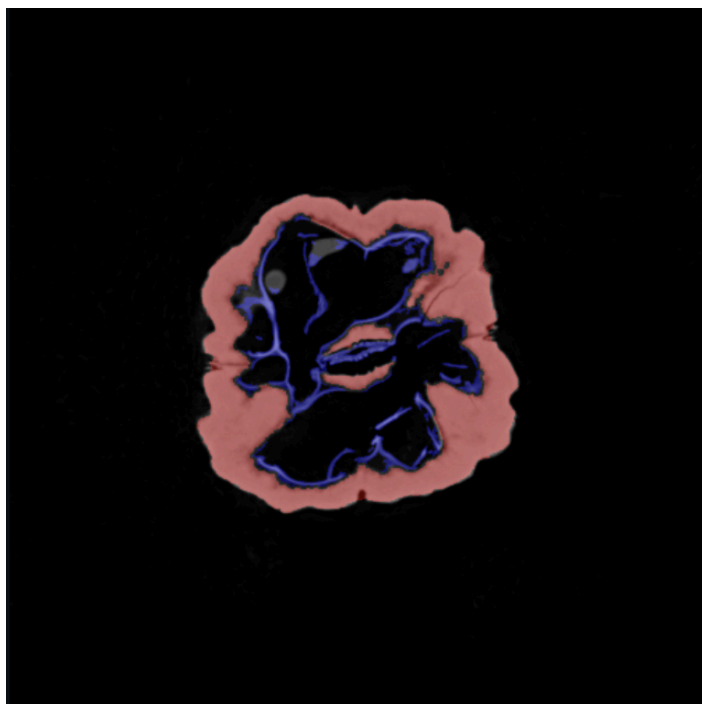


Рис. 32, Близкий к крайним срез ядра (75 по оси Z), где из-за излишней эрозии были удалены маркеры

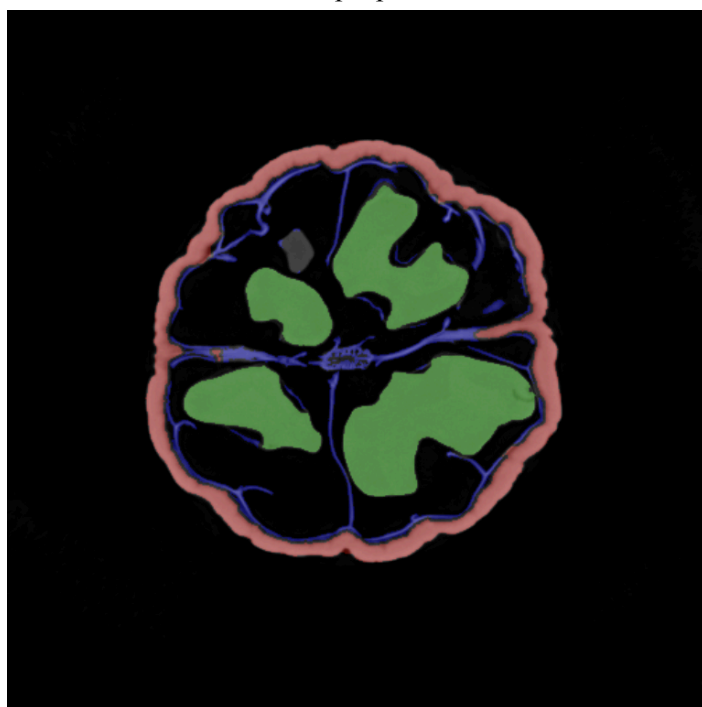


Рис. 33, Срез с существенной разницей между фрагментами ядра в размерах
 При попытках уменьшения количества итераций эрозии в маркеры ядра часто могут попадать широкие части перегородок, близкие к ядрам. При понижении порога фильтрации на дистанционном преобразовании, также в маску часто начинают попадать перегородки.

Неточности в полигональных сетках

На примере сетки ядра можно обнаружить следующие дефекты

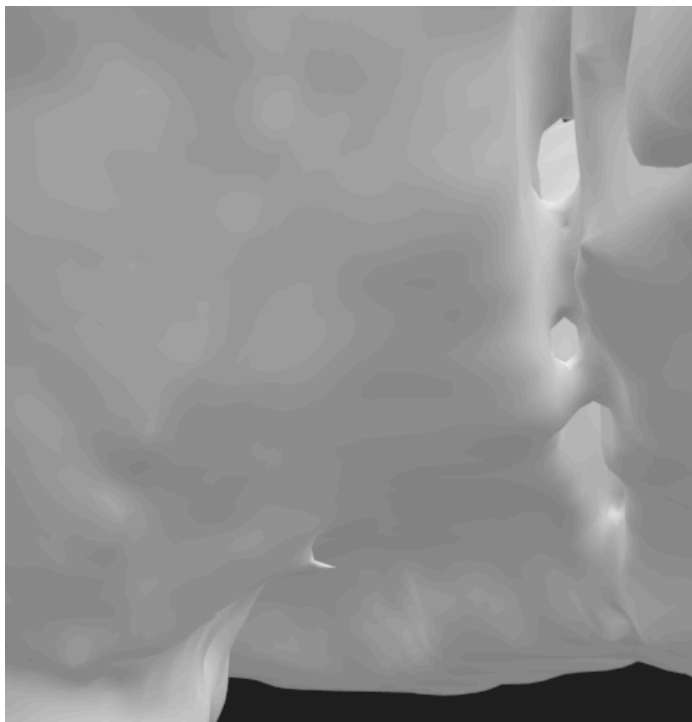


Рис. 34, Дефект сросшихся частей ядра

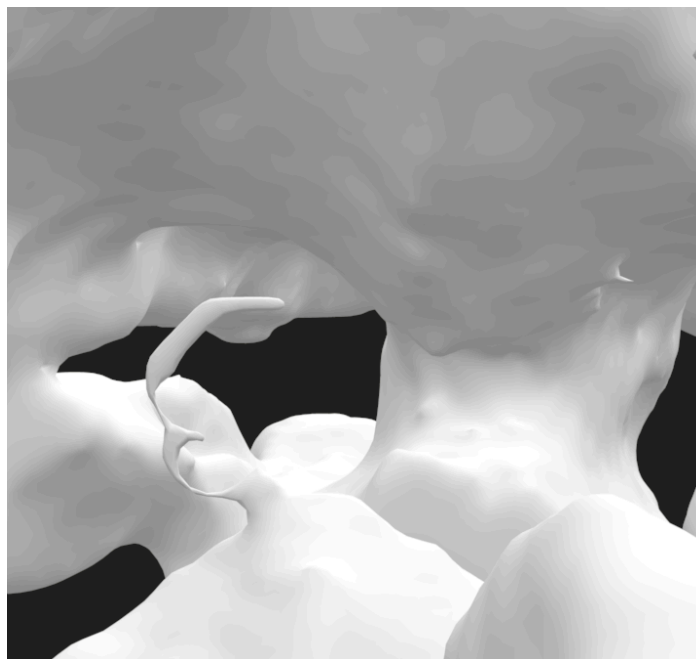


Рис. 35, Дефект торчащих из ядра сегментов

Данные дефекты вероятнее всего являются следствием ошибок, которые были на этапе работы алгоритма *Marching cubes*, а также малых погрешностей в масках сегментации, полученных на выходе из *Segmentator*.